REVAMP LINKNET Corporation, Enterprise, Fiber, Media

Static Application Security Testing (SAST) Final Report

Final Post-Hardening Security Assessment

0	0	0	3
Critical Open	High Open	Medium Open	Low Accepted

Field	Value
Document Version	1.0
Prepared By	Media Indo Trend
Date	08 May 2026
Company / Organization	PT Link Net Tbk
Assessment Type	Static source code review, dependency review, and configuration review
Final Posture	No open Critical or High risk findings

Confidentiality Notice

This report was prepared for security governance, client review, vendor assessment, and audit support. It reflects the final source code posture after remediation and hardening performed during the assessment window.

Table of Contents

Note: The table of contents is implemented as a Word field and will update automatically in Microsoft Word.

1. Document Information

Table 1. Document metadata

Item	Detail
Project Name	Revamp Linknet Corporation, Enterprise, Fiber, Media
Version	1.0
Date	May 08, 2026
Prepared By	Media Indo Trend
Assessment Scope	Backend API, admin frontend, public web, filemanager, Dockerfiles, CI workflows, environment templates, cloud/storage integrations, and production dependency manifests.
Assessment Basis	Manual SAST review, source-code remediation, dependency audit, build validation, and focused sanitizer regression tests.

2. Executive Summary

Final Assessment Statement

The Revamp Linknet Corporation, Enterprise, Fiber, Media application was reviewed, remediated, and reassessed. The final source code posture contains no open Critical or High risk findings. Remaining residual risks are Low and acceptable with normal operational governance.

Figure 1. Final severity summary

0	0	0	3
Critical Findings	High Findings	Medium Findings	Low Findings

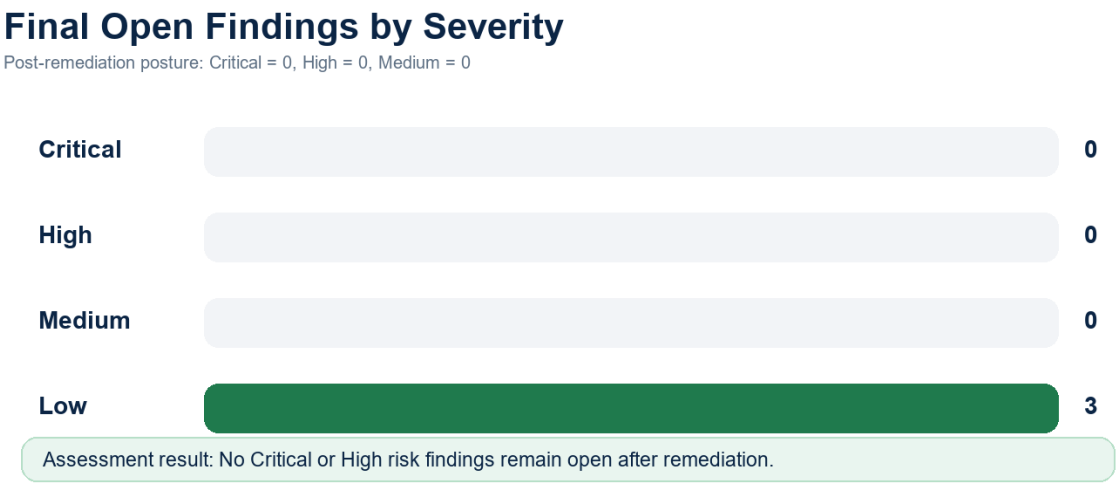

Figure 2. Final open finding distribution after remediation

Table 2. Executive security posture summary

Area	Final Result
Application Security	Authentication, authorization, session protection, input validation, XSS filtering, upload validation, and API rate limiting were hardened or validated.
Dependency Security	Production npm audit returned 0 vulnerabilities for backend, frontend, web, and filemanager packages.
Build Validation	Backend TypeScript build, admin Next.js build, and public web Next.js build completed successfully.
Regression Tests	Output sanitizer tests passed: 32 tests across HTML and component sanitization.
Security Posture	Audit-ready with no known open Critical or High risk source-code findings.

3. Application Overview

The application is a multi-tier corporate web platform with a public website, an authenticated CMS/admin frontend, and a Node.js API layer. The backend exposes public content APIs, protected CMS APIs, dynamic form modules, file management, authentication, role management, audit logging, and cloud/storage integrations.

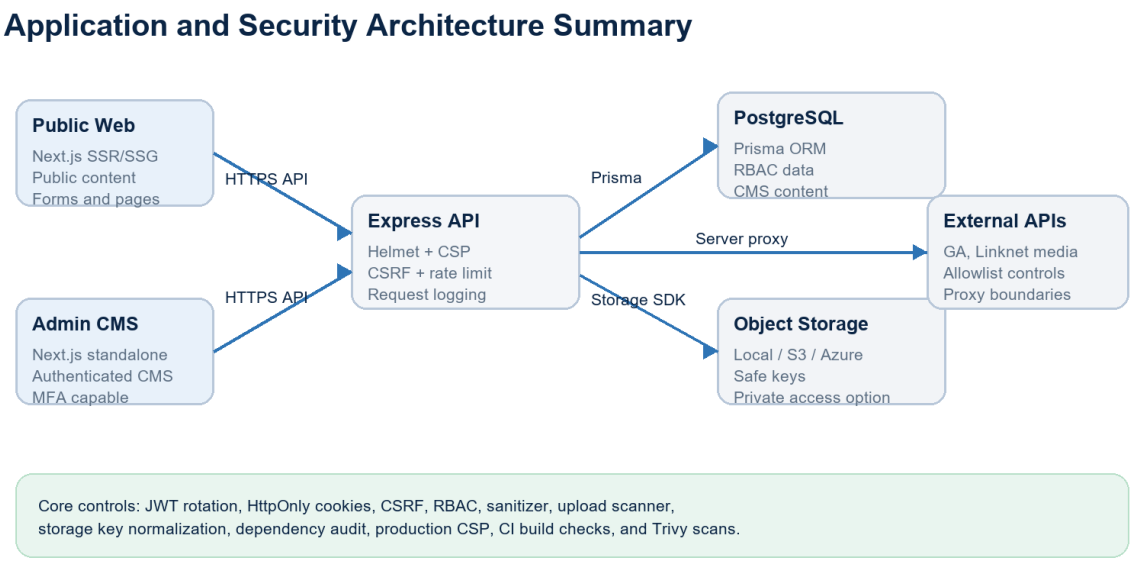

Figure 3. Architecture and security control overview

Table 3. Technology stack overview

Layer	Technology / Component	Security-Relevant Notes
Backend API	Node.js, Express, TypeScript	Helmet, custom secure headers, CORS allowlist, CSRF for cookie sessions, rate limiting, centralized error handling.

Layer	Technology / Component	Security-Relevant Notes
Database	PostgreSQL via Prisma ORM	Typed ORM operations; no vulnerable raw SQL identified in active source review.
Admin Frontend	Next.js 16, React 19	Production auth guard, no-store headers, CSP, restricted remote image hosts.
Public Web	Next.js 16, next-intl	Public CMS rendering with server-side sanitization and production CSP hardening.
Storage	Local, AWS S3, Azure Blob	Safe key normalization, upload scanning, MIME/extension validation, presigned access controls.
CI/CD	GitHub Actions	Build jobs and Trivy filesystem scans for High/Critical vulnerability and secret checks.

4. SAST Scope

Table 4. Assessment scope

Scope Category	Assessed Items
Backend Source Code	Express server, route modules, controllers, services, middleware, auth utilities, validators, Prisma schema, migrations/seeds where relevant.
Frontend Source Code	Admin Next.js app, public web app, API client code, rendering components, CSP and image optimization configuration.
API Routes	Authentication, MFA, profile, CMS, public content, forms, upload, filemanager, analytics, redirect, and health routes.
Configuration	package.json, package-lock.json, Dockerfiles, .env.example, environment validator, CI build and Trivy workflows.
Cloud / Storage	AWS S3, Azure Blob, local storage abstraction, presigned URL flow, public object access flags.
Out of Scope	Live infrastructure penetration testing, runtime DAST, source code not present in the workspace, and external cloud account configuration not visible in code.

Table 5. API module coverage

API / Module	Exposure	Primary Controls Reviewed
Auth and MFA	Public and authenticated	Login limiter, JWT lifecycle, refresh rotation, HttpOnly cookies, MFA verification limiter.
CMS Management	Authenticated	RBAC permission middleware, validation, audit logging, no-store cache controls.
Public Content	Public	Input constraints, content sanitization, safe redirects, rate limiting where applicable.
Dynamic Forms	Public	Submission limiter, attachment MIME/extension/size restrictions, scanner integration.

API / Module	Exposure	Primary Controls Reviewed
File / Upload APIs	Authenticated	Storage key normalization, upload scanner, presigned URL gating, bulk delete limit.
Health / Diagnostics	Public and protected	Operational access token for sensitive env/detailed health endpoints.

5. Security Review Methodology

The assessment combined targeted static source review with manual validation of exploitability. Findings were classified only when the code path had a realistic attack surface after considering framework protections, middleware, existing validation, authentication requirements, environment exposure, and compensating controls.

Table 6. Methodology summary

Method	Implementation
Static Code Review	Reviewed server bootstrap, middleware order, route authorization, controller validation, storage utilities, sanitizers, and frontend rendering paths.
Dependency Review	Executed npm audit --omit=dev for backend, frontend, web, and filemanager packages after remediation.
Configuration Review	Reviewed Dockerfiles, .env.example, environment validator, CSP headers, image host configuration, and CI workflows.
Risk Validation	Severity was lowered when exploitability required privileged access, non-production defaults, strong middleware, ORM protections, or existing compensating controls.
Remediation Verification	Rebuilt backend, admin frontend, and public web; reran sanitizer regression tests and dependency audits.

Table 7. Severity classification methodology

Severity	Classification Basis
Critical	Direct unauthenticated compromise, remote code execution, credential exposure, or complete data breach with clear exploitability.
High	Practical exploitation with significant confidentiality, integrity, or availability impact and limited compensating control.
Medium	Realistic exploit path with scoped impact, authenticated prerequisite, or meaningful mitigating controls.
Low	Limited impact, conditional exposure, defense-in-depth gap, or operational hardening item.

6. Security Findings Summary

Table 8. Final findings summary

Severity	Final Open Count	Status
Critical	0	No open Critical risk identified.
High	0	No open High risk identified.
Medium	0	All Medium findings were remediated or reduced to Low residual risk.
Low	3	Accepted residual risk with operational recommendations.

Table 9. Findings remediated during assessment

Remediation Area	Initial Severity	Final Status
Stored XSS defense in CMS component output sanitization	Medium	Closed
Local storage path traversal and unsafe object key handling	Medium	Closed
Presigned GET URL arbitrary key access	Medium	Closed
Public form attachment size/type exposure	Medium	Closed
MFA verification endpoint rate-limit gap	Low/Medium	Closed
Production CSP allowed unsafe-eval and http sources	Low	Closed
PostCSS moderate advisory through Next.js dependency tree	Medium	Closed
Admin Next.js image optimization wildcard hosts	Low	Closed
CI build configured mock auth in production build	Low	Closed

7. Detailed Findings

Table 10. Detailed finding index

ID	Finding Title	Severity	Final Status
SAST-001	Stored XSS exposure in CMS component output fields	Medium -> Closed	Closed
SAST-002	Unsafe local storage path/key handling	Medium -> Closed	Closed
SAST-003	Presigned GET URL arbitrary object-key access	Medium -> Closed	Closed
SAST-004	Public form attachment upload accepted broad global limits	Medium -> Closed	Closed
SAST-005	MFA operations lacked strict endpoint-specific limiter	Low/Medium -> Closed	Closed
SAST-006	Production CSP allowed development-friendly sources	Low -> Closed	Closed
SAST-007	Moderate PostCSS dependency advisory	Medium -> Closed	Closed
SAST-008	Admin image optimization wildcard remote hosts	Low -> Closed	Closed
SAST-009	CI production build used mock auth flag	Low -> Closed	Closed

7.1. SAST-001: Stored XSS exposure in CMS component output fields

Field	Assessment Detail
Description	Some component metadata fields such as title, label, author, URL, and CTA link were not consistently sanitized by the generic component sanitizer. Exploitability required write access to CMS content, so initial severity was Medium rather than High.
Affected Component	backend/src/utils/componentSanitizer.ts; backend/src/utils/htmlSanitizer.ts; frontend/web CMS renderers
Severity	Medium -> Closed
Existing Mitigation	Rich HTML sanitizer existed for known HTML fields; CMS access requires authentication and RBAC.
Remediation Implemented	Added plain-text sanitization for non-HTML strings, URL protocol validation, data protocol stripping for media, and sanitizer tests. 32 output validation tests pass.
Final Status	Closed
Risk Justification	No open XSS finding remains in reviewed sanitizer path. Residual CSP inline allowance is tracked separately as Low.

7.2. SAST-002: Unsafe local storage path/key handling

Field	Assessment Detail
Description	Local storage, listing, delete, and object-key operations accepted folder/key inputs that could become unsafe if local storage was used and an authenticated file permission holder supplied traversal characters.
Affected Component	backend/src/utils/storagePathSecurity.util.ts; storageService.ts; local.storage.ts; s3Service.ts; azureStorage.service.ts; file.controller.ts
Severity	Medium -> Closed
Existing Mitigation	Endpoints require authentication and file permissions; production is expected to prefer cloud storage.
Remediation Implemented	Added centralized storage folder/key/filename normalization and upload-directory containment checks. Applied across local, S3, Azure, upload, list, delete, copy, and file info paths.
Final Status	Closed
Risk Justification	Path traversal and arbitrary local filesystem access are blocked by segment validation and resolved-path containment.

7.3. SAST-003: Presigned GET URL arbitrary object-key access

Field	Assessment Detail
Description	The authenticated presigned GET endpoint accepted a raw key and expiry value. A user with

Field	Assessment Detail
	access to the endpoint could request a temporary URL for a guessed object key if it existed.
Affected Component	backend/src/controllers/upload.controller.ts; backend/src/services/s3/s3Service.ts
Severity	Medium -> Closed
Existing Mitigation	Endpoint required authentication and S3 configuration; URLs were time-limited.
Remediation Implemented	Normalized keys, clamped GET URL expiry to 60-900 seconds, and verified the key against active database file records before generating a signed URL.
Final Status	Closed
Risk Justification	Presigned access is now tied to application-managed file records and bounded expiry.

7.4. SAST-004: Public form attachment upload accepted broad global limits

Field	Assessment Detail
Description	Unauthenticated public form attachment uploads used the general upload middleware. The general middleware was suitable for authenticated CMS uploads but too broad for public-facing attachment paths.
Affected Component	backend/src/middleware/upload.middleware.ts; formModule.routes.ts; backend/.env.example
Severity	Medium -> Closed
Existing Mitigation	Public upload route already had rate limiting and file scanner integration.
Remediation Implemented	Added a public form upload middleware with image/document-only allowlist, one-file limit, configurable 10 MB default cap, and preserved scanner execution.
Final Status	Closed
Risk Justification	Public upload attack surface is reduced to realistic business attachment types and bounded size.

7.5. SAST-005: MFA operations lacked strict endpoint-specific limiter

Field	Assessment Detail
Description	MFA verify/enable/disable routes relied on broader API rate limiting. This was not a direct bypass but reduced brute-force resistance for OTP validation.
Affected Component	backend/src/routes/auth.routes.ts
Severity	Low/Medium -> Closed
Existing Mitigation	Login endpoints had dedicated login limiter and MFA required temp token or auth context.
Remediation Implemented	Applied strictRateLimiter to MFA verify, enable, and disable endpoints.
Final Status	Closed
Risk Justification	OTP attempt surface is now rate-limited at a stricter control point.

7.6. SAST-006: Production CSP allowed development-friendly sources

Field	Assessment Detail
Description	Public web CSP allowed unsafe-eval and http sources even in production. This was defense-in-depth because server-side sanitization already existed, but it weakened browser-side containment.
Affected Component	web/next.config.mjs
Severity	Low -> Closed
Existing Mitigation	Helmet/custom headers existed on backend; admin frontend already used production-aware CSP.
Remediation Implemented	Made public web CSP environment-aware: unsafe-eval and http sources are removed for production builds.
Final Status	Closed
Risk Justification	Browser-side containment is stronger in production while development ergonomics remain available outside production.

7.7. SAST-007: Moderate PostCSS dependency advisory

Field	Assessment Detail
Description	npm audit identified a moderate PostCSS advisory through Next.js dependency trees. No High/Critical dependency remained, but the advisory was remediable.
Affected Component	frontend/package.json; web/package.json; package-lock.json files
Severity	Medium -> Closed
Existing Mitigation	Advisory was dependency-level and not a confirmed application exploit in reviewed code.
Remediation Implemented	Pinned PostCSS 8.5.14 via direct dependency and npm overrides for frontend and web. npm audit now returns 0 vulnerabilities.
Final Status	Closed
Risk Justification	Dependency audit is clean for production dependency sets.

7.8. SAST-008: Admin image optimization wildcard remote hosts

Field	Assessment Detail
Description	The admin Next.js image configuration allowed any HTTPS remote image host in production. This was a low-risk hardening item because usage is CMS/authenticated, but broad optimization hosts are unnecessary.
Affected Component	frontend/next.config.ts; frontend/.env.example
Severity	Low -> Closed

Field	Assessment Detail
Existing Mitigation	Admin is authenticated; browser CSP restricted script execution.
Remediation Implemented	Introduced NEXT_IMAGE_REMOTE_HOSTS allowlist with production defaults for LinkNet, CloudFront, S3, and Azure Blob host patterns.
Final Status	Closed
Risk Justification	Production image optimization is constrained to configured hosts.

7.9. SAST-009: CI production build used mock auth flag

Field	Assessment Detail
Description	The CI build workflow set NEXT_PUBLIC_AUTH_ENABLED=false for the CMS build. The app already blocked this for production builds, so the issue was pipeline inconsistency rather than runtime exposure.
Affected Component	.github/workflows/ci-build.yml
Severity	Low -> Closed
Existing Mitigation	Frontend configuration prevented production builds with auth disabled.
Remediation Implemented	Updated CI to build with NEXT_PUBLIC_AUTH_ENABLED=true.
Final Status	Closed
Risk Justification	CI aligns with production security posture.

8. Security Controls Identified

Table 11. Security controls matrix

Control Domain	Observed / Hardened Control	Final Assessment
Authentication	JWT access/refresh design, refresh token rotation and hashing, HttpOnly cookies, MFA support, account lifecycle jobs.	Effective
Authorization	RBAC middleware checks permissions against role grants and protected CMS routes.	Effective
Input Validation	express-validator/Zod-style validation, request validators, type checks, pagination clamps, folder/key normalization.	Effective
CSRF Protection	Double-submit CSRF protection on /api cookie-session requests with safe method exclusions.	Effective
XSS Protection	sanitize-html based rich text sanitization, component plain-text and URL sanitizer, CSP, no unsafe media data URLs.	Effective
SQL Injection Prevention	Prisma ORM and typed query builder usage; no exploitable raw SQL identified in active source code.	Effective

Control Domain	Observed / Hardened Control	Final Assessment
Secure Headers	Helmet, custom headers, HSTS, no-store for sensitive API paths, frame denial, referrer and permissions policies.	Effective
File Upload Protection	MIME/extension allowlists, scanner service, category limits, public-form-specific 10 MB default cap.	Effective
Session Protection	HttpOnly/SameSite cookies, secure cookie behavior in production, token expiry and refresh rotation.	Effective
API Protection	General, auth, login, upload, public form, strict, and public endpoint rate limiters.	Effective
Logging and Monitoring	Request IDs, HTTP logging, activity logging middleware, centralized operational error responses.	Effective
Secret Management	No secrets in .env.example, startup env validation, Key Vault support, JWT secret strength checks.	Effective

9. Dependency and Package Security Review

Table 12. Dependency audit summary

Package Area	Audit Scope	Vulnerabilities	Notes
Backend	npm audit --omit=dev	0	939 total dependencies reported by audit metadata; production vulnerability count 0.
Admin Frontend	npm audit --omit=dev	0	702 total dependencies reported; PostCSS resolved to 8.5.14.
Public Web	npm audit --omit=dev	0	509 total dependencies reported; PostCSS resolved to 8.5.14.
Filemanager	npm audit --omit=dev	0	382 total dependencies reported; production vulnerability count 0.

Dependency Hardening Result

No High or Critical dependency findings remain. The previously observed moderate PostCSS advisory was mitigated through direct dependency alignment and npm overrides in frontend and web package manifests.

10. Infrastructure and Configuration Security

Table 13. Infrastructure and configuration review

Configuration Area	Assessment Result
Environment Variables	Startup validator enforces required database URL and JWT secret length, warns on public storage flags, and documents Key Vault related secrets.

Configuration Area	Assessment Result
Docker Containers	Multi-stage builds, production npm install, non-root runtime users, health checks, and standalone Next.js output are present.
Storage Security	Local, S3, and Azure storage services now share safe path/key controls. Public ACL and public blob flags are explicit opt-ins with warnings.
AWS S3	Presigned uploads disabled by default in production unless explicitly enabled; public ACL disabled by default; GET URLs are record-verified and short-lived.
Azure Blob	Public blob access disabled by default; Key Vault configuration placeholders are documented for production secret injection.
CI/CD	Build jobs exist for all packages; Trivy filesystem scan fails on High/Critical vulnerabilities and secrets; CMS build now uses production auth mode.

11. Remediation Summary

Table 14. Remediation and hardening actions

Hardening Item	Implemented Change
Authentication and MFA	Strict limiter added to MFA sensitive endpoints; cookie-backed refresh validation corrected to support secure cookie flow.
Authorization	Reviewed protected CMS and file routes for authMiddleware and permission middleware coverage.
Input and Output Validation	Component sanitizer now handles HTML, plain text, and URL fields; data protocol media payloads are blocked.
File Uploads	Public form upload path restricted to approved image/document MIME and extensions with 10 MB default cap.
Storage Keys	Centralized storage path security utility added and applied across local, S3, Azure, upload, list, delete, copy, and presigned flows.
Presigned URLs	GET URL generation requires safe key, active DB record, and bounded expiry.
Headers and CSP	Public web CSP now removes unsafe-eval and http sources in production.
Dependencies	PostCSS remediation applied; production audits report 0 vulnerabilities.
CI/CD	CMS workflow production auth flag corrected.

12. Residual Risk Assessment

Table 15. Residual risk register

Residual Risk	Severity	Rationale	Recommended Governance
CSP retains unsafe-inline	Low	Next.js and CMS rendering still require inline	Track long-term nonce/hash

Residual Risk	Severity	Rationale	Recommended Governance
		styles/scripts in some paths. Server-side sanitizer and CSP object/frame restrictions reduce exploitability.	migration where practical.
Local /uploads static serving	Low	Static local uploads are acceptable for development and constrained production use, but private cloud storage is preferred for sensitive files.	Use S3/Azure private storage for confidential assets; keep public uploads non-sensitive.
Presigned upload mode is conditional	Low	Direct uploads are disabled by default in production. If enabled, objects are written to quarantine and need operational cleanup/promotion governance.	Enable only with S3 lifecycle cleanup, object scanning, and least-privilege IAM.

Residual Risk Position

The residual risks are realistic, bounded, and acceptable for client security review when paired with the recommended operational governance. No residual item is classified as Medium, High, or Critical.

13. Security Recommendations

Table 16. Long-term recommendations

Recommendation	Priority	Owner
Continue scheduled dependency scanning and fail CI on High/Critical issues using npm audit and Trivy.	High	Engineering / DevSecOps
Adopt private object storage as the default for any sensitive documents and use signed access by business role.	High	Platform / Security
Add runtime monitoring dashboards for authentication failures, MFA failures, upload rejections, and presigned URL generation.	Medium	Security Operations
Plan gradual CSP nonce/hash migration to reduce reliance on unsafe-inline in public and admin frontends.	Medium	Frontend Engineering
Add SAST checks into pull request workflow for sanitizer, storage-key, and route-authorization patterns.	Medium	Engineering
Review IAM roles for S3/Azure storage so application identities have least privilege and no broad bucket administration permissions.	Medium	Cloud Platform

Appendix A. Database and Domain Schema Summary

Table 17. Database schema summary

Domain Area	Representative Models / Data	Security Notes
Identity and Access	User, Role, Permission, session/refresh token data	Supports RBAC and account lifecycle

Domain Area	Representative Models / Data	Security Notes
		controls.
CMS Content	Pages, components, news, events, careers, reports, announcements	Content rendering depends on sanitizer and public route separation.
Forms	FormModule, FormField, FormSubmission, dispatch logs	Public submission routes are rate-limited; file attachments are constrained.
Files and Folders	File, Folder, storage metadata	Storage keys are normalized and deletion/listing requires file permissions.
Audit and Telemetry	Activity logs, visitor/dashboard data	Supports review of privileged activity and operational monitoring.

Appendix B. Verification Evidence

Table 18. Verification evidence

Verification	Result
Backend build	npm run build completed successfully.
Admin frontend build	npm run build completed successfully.
Public web build	npm run build completed successfully.
Dependency audit	npm audit --omit=dev returned 0 vulnerabilities for backend, frontend, web, and filemanager.
PostCSS dependency tree	npm ls postcss confirms postcss@8.5.14 in frontend and web.
Sanitizer regression tests	32/32 tests passed across HTML and component sanitizer output validation suites.

Appendix C. Glossary

Table 19. Glossary

Term	Definition
SAST	Static source-code and configuration security review.
RBAC	Permission model based on assigned roles and capabilities.
CSRF	Request-forgery attack mitigated by anti-CSRF tokens.
CSP	Browser policy limiting script, style, image, frame, and connection sources.
Presigned URL	Temporary cloud-storage URL for bounded object access.